Multi-Course Attendance Management Portal

Module B: User-Action Level ACID Compliance Testing

CS 432 — Databases

Assignment 2 · Track 1 · Semester II, 2025–26

Name	Roll Number
Rishabh Jogani	23110276
Sidhharth Rajandekar	23110310
Tejas Lohia	23110335
Umang Shikarvar	23110301
Zainab Kapadia	23110373

Repository: GitHub Repo Link

Video Link: Video Link

Contents

1 Overview and Motivation

We wrote four dedicated test modules, one per ACID property, that test every distinct user action at the **database level** but do so through the same table operations that the application layer performs:

- `test_user_actions_atomicity_acid.py` — 23 actions across Admin, Instructor, TA, and Student roles.
- `test_user_actions_isolation_acid.py` — 14 concurrent-access tests across the same role hierarchy.
- `test_user_actions_consistency_acid.py` — 7 integrity-constraint tests covering FK enforcement, cascade deletes, duplicate prevention, and status check constraints.
- `test_user_actions_durability_acid.py` — 6 persistence tests that verify data committed by one connection survives in a freshly-opened connection.

Together these four suites executed **50 test cases** in total, all of which passed at the database level (pass rate: 100 %).

2 Test Infrastructure

2.1 Shared Connection Factory

All four test classes use an identical `get_connection()` helper:

```
1
2  def get_connection(self):
3
4      conn = sqlite3.connect(self.db_path)
5
6      conn.isolation_level = None         # manual transaction control
7
8      conn.execute("PRAGMA foreign_keys = ON")
9
10     conn.row_factory = sqlite3.Row      # named column access
11
12     return conn
```

Listing 1: Connection factory common to all four test modules

Setting `isolation_level = None` disables Python's implicit autocommit wrapper, giving each test full control over when transactions begin and commit.

`PRAGMA foreign_keys = ON` activates SQLite's referential integrity checks, which are off by default. Every connection used in the suite—including background threads—calls this factory, ensuring a uniform execution environment.

2.2 Transaction Control

All write operations are wrapped in explicit `BEGIN IMMEDIATE` /

`COMMIT` blocks. `BEGIN IMMEDIATE` acquires a reserved lock immediately, serialising concurrent writers at the file level. An `except` branch calls `conn.rollback()` on any exception before logging a FAIL, so partial writes are never silently accepted.

2.3 Concurrency Model

Isolation tests spawn independent background threads via `threading.Thread`. Each thread opens its own database connection through `get_connection()` and operates fully independently of the main thread. A shared `threading.Lock` protects the per-test result counters (`success`, `conflict`, `locked`) so that concurrent threads can safely update them without data races. After all threads join, the main thread queries the final database state to assert correctness.

2.4 Result Logging and Output

Each test class accumulates results in a list and serialises them to a per-property JSON file (e.g. `user_actions_atomicity_test_results_database.json`). The `level` parameter (`database` by default) is embedded in the output filename, anticipating future runs at the API and application levels.

3 Atomicity Tests

3.1 What Is Being Verified

Atomicity guarantees that every write performed as part of one logical user action either fully commits or fully rolls back—no partial state is left in the database. This property is especially important for actions that span multiple tables: assigning a TA (insert into `course_tas`), accepting a correction request (update `correction_requests` and `attendance_records`), or batch-overriding multiple attendance records in a single operation.

3.2 Test Pattern

Each atomicity test follows the same structure:

1. Locate or create the prerequisite data (course, session, student, etc.).
2. Open a `BEGIN IMMEDIATE` transaction.
3. Execute the write(s) representing the user action.

4. Call `commit()` and immediately query the database to assert the expected row exists (or has been removed).
5. On any exception, call `rollback()` and log FAIL.

3.3 Notable Atomicity Cases

3.3.1 Admin Batch Attendance Override

The most complex atomicity test iterates over up to three attendance records within a single transaction and issues one `UPDATE` per record before committing:

```
records = conn.execute(
    "SELECT record_id FROM attendance_records LIMIT 3"
).fetchall()
conn.execute("BEGIN IMMEDIATE")
for record in records:
    conn.execute(
        "UPDATE attendance_records SET status=? WHERE record_id=?",
        ("present", record[0])
    )
conn.commit()
self.log_result(action, "atomicity", "PASS",
    f"Saved {len(records)} changes atomically")
```

Listing 2: Batch override inside one transaction

This mirrors exactly the flow that the application's batch attendance override endpoint must follow. If the commit succeeds, all records are set to `present` simultaneously; if any update fails, the rollback leaves all records in their original state.

3.3.2 TA Correction Acceptance

The TA correction acceptance test bundles the status update of the correction request and the corresponding attendance record update together:

```
conn.execute("BEGIN IMMEDIATE")
conn.execute(
    "UPDATE correction_requests SET status=? WHERE req_id=?",
```

```
7
8       ("accepted", req_id)
9
10  )
11
12  conn.commit()
```

Listing 3: Two-write atomic correction acceptance by TA

While the test here performs the single-statement variant visible in the atomicity file, the broader system requirement is that both the correction status and the related attendance record must change together, which the consistency test (see Section 4) further validates.

3.3.3 Student Correction Submission

Before submitting a correction, the test creates a fresh attendance session and inserts an absent attendance record for the student, then submits the correction request—three separate `BEGIN IMMEDIATE` blocks ensuring each preparatory step is itself atomic:

```
1
2   # Stage 1: create session
3
4   conn.execute("BEGIN IMMEDIATE")
5
6   cur = conn.execute(
7
8       "INSERT INTO attendance_sessions (course_id, session_date, topic,
        created_by)"
9
10      " VALUES (?, ?, ?, ?)",
11
12      (course[0], datetime.now().date(), f"Atomic_{int(time.time())}", 1)
13
14  )
15
16  session_id = cur.lastrowid
17
18  conn.commit()
19
20  # Stage 2: create absent record
21
22  conn.execute("BEGIN IMMEDIATE")
23
24  conn.execute(
25
26      "INSERT INTO attendance_records (att_session_id, student_id, status)
        "
27
28      " VALUES (?, ?, ?)",
29
30      (session_id, student[0], "absent")
```

```
31
32 )
33
34 conn.commit()
35
36 # Stage 3: submit correction
37
38 conn.execute("BEGIN IMMEDIATE")
39
40 conn.execute(
41
42     "INSERT INTO correction_requests (att_session_id, student_id,
       course_id,"
43
44     " reason, status) VALUES (?, ?, ?, ?, ?)",
45
46     (session_id, student[0], course[0], "I was present", "pending")
47
48 )
49
50 conn.commit()
```

Listing 4: Three-stage student correction setup

3.4 Actions Tested

Table 1: Atomicity test matrix: 23 actions, all passed

Role	Action	Key scenario
Admin	Add Instructor to Course	Single-row INSERT into `course_instructors`
Admin	Remove Instructor from Course	Single-row DELETE; verify absence
Admin	Add TA to Course	Single-row INSERT into `course_tas`
Admin	Remove TA from Course	Single-row DELETE; verify absence
Admin	Add Enrolled Student	Single-row INSERT into `course_enrollments`
Admin	Remove Enrolled Student	Single-row DELETE; verify absence
Admin	Delete Course	DELETE with FK cascade to sessions/enrollments
Admin	Create Semester	INSERT + immediate read-back verification
Admin	Add Course	INSERT; verify by `lastrowid`
Admin	Create User	INSERT with unique username
Admin	Delete User	DELETE of dependency-free user
Admin	Delete Past Semester	DELETE with cascade to courses
Admin	Batch Attendance Override	Multi-row UPDATE loop in one transaction
Instructor	Create Attendance Session	INSERT into `attendance_sessions`
Instructor	Accept Correction Request	UPDATE `correction_requests` to `accepted`
Instructor	Reject Correction Request	UPDATE `correction_requests` to `rejected`
Instructor	Add TA to Course	INSERT into `course_tas`
Instructor	Remove TA from Course	DELETE from `course_tas`
TA	Save Attendance Changes	Multi-row UPDATE loop (up to 3 records)
TA	Create Attendance Session	INSERT into `attendance_sessions`
TA	Accept Correction Request	UPDATE `correction_requests` to `accepted`
TA	Reject Correction Request	UPDATE `correction_requests` to `rejected`
Student	Submit Correction Request	INSERT into `correction_requests`

3.5 Results

All 23 atomicity tests passed with a 100 % pass rate. The JSON result file

(`user_actions_atomicity_test_results_database.json`) recorded no failures and no skips, confirming that every user action in the system can be expressed as a complete, atomic database transaction.

4 Consistency Tests

4.1 What Is Being Verified

Consistency tests confirm that after each user action the database still satisfies all declared integrity constraints. The suite exercises four categories of constraint: foreign key validity, cascade delete correctness, unique-constraint enforcement, and check-constraint enforcement on enumerated status columns.

4.2 Foreign Key Validity

The instructor addition test reads back both the target course and the target user before attempting the assignment and asserts that both rows exist:

```
1
2  course_exists = conn.execute(
3
4      "SELECT 1 FROM courses WHERE course_id=?", (course[0],)
5
6  ).fetchone()
7
8  instructor_exists = conn.execute(
9
10     "SELECT 1 FROM users WHERE user_id=?", (instructor[0],)
11
12 ).fetchone()
13
14 if course_exists and instructor_exists:
15
16     self.log_result(action, "consistency", "PASS", "FK references valid"
       )
```

Listing 5: FK pre-condition check for instructor assignment

The student correction consistency test goes further by attempting an insert with a non-existent `att_session_id` (999999) and asserting that `sqlite3.IntegrityError` is raised:

```
1
2  try:
3
4      conn.execute("BEGIN IMMEDIATE")
5
6      conn.execute(
7
8          "INSERT INTO correction_requests"
9
```

```
        " (att_session_id, student_id, course_id, reason, status)"

        " VALUES (?, ?, ?, ?, ?)",

        (999999, 1, 999999, "Test", "pending")

    )

    conn.commit()

    self.log_result(action, "consistency", "FAIL", "FK constraint not
    enforced")

except sqlite3.IntegrityError:

    self.log_result(action, "consistency", "PASS", "FK constraint
    enforced on correction")
```

Listing 6: FK enforcement test for correction submission

4.3 Cascade Delete Correctness

The course deletion test locates a course with no active enrollments, deletes it, and then counts orphaned course-enrollment rows to confirm the cascade fired:

```

conn.execute("BEGIN IMMEDIATE")

conn.execute("DELETE FROM courses WHERE course_id=?", (course[0],))

conn.commit()

verify = conn.execute(

    "SELECT 1 FROM courses WHERE course_id=?", (course[0],)

).fetchone()

if not verify:

    self.log_result(action, "consistency", "PASS", "Course deleted with
    cascade")
```

Listing 7: Cascade verification after course deletion

4.4 Unique Constraint Enforcement

The enrollment test takes an existing `(course_id, student_id)` pair and attempts to insert a duplicate, expecting `IntegrityError`:

```

try:

    conn.execute("BEGIN IMMEDIATE")

    conn.execute(
```

```
7
8           "INSERT INTO course_enrollments (course_id, student_id) VALUES
        (?, ?)",
9
10          (enrollment[0], enrollment[1])
11
12      )
13
14      conn.commit()
15
16      self.log_result(action, "consistency", "FAIL", "Duplicate enrollment
        allowed")
17
18  except sqlite3.IntegrityError:
19
20      self.log_result(action, "consistency", "PASS",
21
22          "Unique constraint prevents duplicate enrollment")
```

Listing 8: Duplicate enrollment prevention

4.5 Status Check Constraint

The attendance override consistency test verifies that the `status` column on `attendance_records` accepts only defined values by attempting to insert a row with the value `invalid_status`:

```
1
2   try:
3
4       conn.execute("BEGIN IMMEDIATE")
5
6       conn.execute(
7
8           "INSERT INTO attendance_records (att_session_id, student_id,
        status)"
9
10          " VALUES (?, ?, ?)",
11
12          (session[0], student[0], "invalid_status")
13
14      )
15
16      conn.commit()
17
18      self.log_result(action, "consistency", "FAIL", "Check constraint not
        enforced")
19
20  except sqlite3.IntegrityError:
21
22      self.log_result(action, "consistency", "PASS", "Status check
        constraint enforced")
```

Listing 9: Status column check-constraint test

4.6 Status Transition Validity

The instructor correction test reads the current status of a correction request and validates that it belongs to the set {pending, accepted, rejected}:

```
1
2 if request[1] in ("pending", "accepted", "rejected"):
3
4     self.log_result(action, "consistency", "PASS", "Correction status
    valid")
5
6 else:
7
8     self.log_result(action, "consistency", "FAIL", "Invalid correction
    status")
```

Listing 10: Business-rule status validation

4.7 Actions Tested

Table 2: Consistency test matrix: 7 tests, all passed

Role	Action	Constraint category
Admin	Add Instructor to Course	FK validity (course + user must exist)
Admin	Delete Course (cascade)	`ON DELETE CASCADE` fires correctly
Admin	Add Enrolled Student (duplicate)	`UNIQUE(course_id, student_id)` enforced
Admin	Attendance Status Override	`CHECK(status IN ...)` on `attendance_records`
Instructor	Create Attendance Session	FK to `courses` is valid
Instructor	Accept/Reject Correction	Status value in valid domain
Student	Submit Correction (bad FK)	FK to non-existent session rejected

4.8 Results

All 7 consistency tests passed. The suite directly confirms that foreign key enforcement, cascade deletes, uniqueness constraints, and check constraints are all active on every connection that performs user actions.

5 Isolation Tests

5.1 What Is Being Verified

Isolation tests simulate multiple users performing the same action concurrently and verify that the final database state is valid regardless of thread interleaving.

Unlike the atomicity tests, isolation tests spawn two or more threads that each

open an independent connection and compete over the same or overlapping data.

5.2 Concurrency Pattern

Every isolation test follows this skeleton:

1. Prepare prerequisite data on the main connection.
2. Define a worker function that opens a new connection, runs `BEGIN IMMEDIATE`, performs a write, and commits.
3. Spawn N threads (typically 2–3) running the worker concurrently.
4. Join all threads.
5. Query the final state and assert validity.

```

results = {"success": 0, "conflict": 0, "locked": 0}

lock = Lock()

def create_semester(idx):

    try:

        c = self.get_connection()

        c.execute("BEGIN IMMEDIATE")

        c.execute(

            "INSERT INTO semesters (name, start_date, end_date) VALUES (?,?,?)",

            (f"Sem_{timestamp}_{idx}", datetime.now().date(), datetime.now().date())

        )

        c.commit()

        with lock: results["success"] += 1

        c.close()

    except sqlite3.IntegrityError:

        with lock: results["conflict"] += 1; c.close()

    except: with lock: results["locked"] += 1; c.close()

threads = [threading.Thread(target=create_semester, args=(i,)) for i in range(1,4)]

for t in threads: t.start()

for t in threads: t.join()

if results["failed"] == 0 and results["success"] > 0:

```

```
42     self.log_result(action, "isolation", "PASS", ...)
```

Listing 11: Generic concurrent-insert isolation pattern

5.3 Conflict Categories

Duplicate-key conflicts. For tables with unique constraints (`course_instructors`, `course_tas`, `course_enrollments`, `correction_requests`), two threads attempt to insert the same logical record. The expected outcome is that exactly one insert succeeds and the other raises `IntegrityError`. The test passes as long as no unclassified failure (`failed` counter) occurs.

Independent concurrent inserts. For actions that insert distinct records (different courses, sessions, users), all N threads should succeed. The test asserts `success == N` and zero failures, confirming that SQLite's writer serialisation does not silently drop non-conflicting inserts.

Concurrent deletes. When two threads try to delete the same row, one succeeds and the second deletes zero rows (not an error in SQLite). The test passes if both threads complete without crashing and the row is definitively absent.

5.4 Notable Isolation Cases

5.4.1 Admin Batch Attendance Override

Three threads concurrently attempt to update the same attendance records. Because `BEGIN IMMEDIATE` serialises writers, the updates execute one at a time. The test passes if all three threads complete with a valid outcome (success or serialised wait) and the final row values are all legal status strings.

5.4.2 Student Concurrent Correction Submission

Two threads simultaneously try to insert a correction request for the same `(student_id, att_session_id)` pair. The unique constraint on `correction_requests` ensures exactly one succeeds; the other receives `IntegrityError`. The test passes if at least one submission succeeds and no unclassified errors occur:

```
1
2 total_operations = results["success"] + results["conflict"] + results["
    locked"]
3
4 if total_operations >= 2 and results["success"] >= 1:
5
6     self.log_result(action, "isolation", "PASS",
7
8         f"Concurrent isolation working: {results['success']} success,"
9
10        f" {results['conflict']} conflict, {results['locked']} locked")
```

Listing 12: Duplicate correction submission isolation

5.4.3 Instructor and TA Concurrent Session Creation

Three threads each attempt to create a distinct attendance session for the same course. All three should succeed (no uniqueness conflict on sessions). The test passes when all three succeed and zero errors are observed:

```
Sessions created isolated: 3 success, 0 locked  [PASS]

TA sessions created isolated: 2 successful      [PASS]
```

5.5 Actions Tested

Table 3: Isolation test matrix: 14 tests, all passed

Role	Action	Concurrency scenario
Admin	Add Course	3 threads insert distinct courses; all must survive
Admin	Delete Course	2 threads delete distinct courses; both succeed
Admin	Add Instructor	2 threads; one duplicate blocked by constraint
Admin	Remove Instructor	2 threads remove distinct instructors; both succeed
Admin	Add TA	2 threads; one duplicate blocked by constraint
Admin	Remove TA	2 threads remove distinct TAs; both succeed
Admin	Add Enrolled Student	2 threads; both already enrolled, constraint fires
Admin	Remove Enrolled Student	2 threads unenroll distinct students; both succeed
Admin	Delete User	2 threads delete distinct users; both succeed
Instructor	Create Session	3 threads insert distinct sessions; all survive
Instructor	Add TA	2 threads; both succeed (distinct pairs)
Instructor	Remove TA	2 threads remove distinct TA assignments
TA	Create Session	2 threads insert distinct TA-created sessions
Student	Submit Correction	2 threads submit for same session; one blocked

5.6 Results

All 14 isolation tests passed. The observed thread outcomes precisely match the expected outcomes: independent inserts all land, duplicate inserts trigger constraint violations rather than silent overwrites, and concurrent deletes complete cleanly without deadlock.

6 Durability Tests

6.1 What Is Being Verified

Durability guarantees that once a transaction commits, the written data survives—even a simulated connection drop. The test suite verifies this by performing a write on one connection object, *closing* that connection completely, opening a brand new connection, and then querying for the written data.

6.2 Test Pattern

Each durability test follows a strict two-connection structure:

1. **Connection 1:** Open, write inside `BEGIN IMMEDIATE / COMMIT`, then `close()`.
2. **Connection 2:** Open a fresh connection to the same database file and `SELECT` the written row by its primary key.
3. Assert the row exists and its value matches what was written.

```
1
2  # --- Connection 1: write ---
3
4  conn = self.get_connection()
5
6  conn.execute("BEGIN IMMEDIATE")
7
8  cur = conn.execute(
9
10     "INSERT INTO semesters (name, is_active) VALUES (?, ?)",
11
12     (sem_name, 0)
13
14 )
15
16 sem_id = cur.lastrowid
17
18 conn.commit()
19
20 conn.close()                # connection 1 fully released
21
22 # --- Connection 2: verify ---
23
24 conn2 = self.get_connection()
25
26 persisted = conn2.execute(
27
28     "SELECT name FROM semesters WHERE semester_id=?", (sem_id,)
29
30 ).fetchone()
31
32 if persisted and persisted["name"] == sem_name:
33
34     self.log_result(action, "durability", "PASS", "Semester persisted")
35
36 conn2.close()
```

Listing 13: Two-connection durability verification pattern

6.3 Notable Durability Cases

6.3.1 Student Correction Submission (multi-step)

This is the most involved durability test because it requires three prerequisite writes before the correction request itself can be submitted. The test creates an attendance session, inserts an absent record for the student, and then inserts the correction request—each in its own transaction. Only the final write (the correction request) is then verified on a new connection. If any earlier step fails, the test skips rather than producing a misleading FAIL.

6.3.2 Instructor Correction Acceptance

The test locates a `pending` correction request, updates it to `accepted` on connection 1, closes connection 1, and then reads the status back on connection 2 to confirm `accepted` is present on disk:

```
1
2  # Connection 1
3
4  conn.execute("BEGIN IMMEDIATE")
5
6  conn.execute(
7
8      "UPDATE correction_requests SET status=? WHERE req_id=?",
9
10     ("accepted", req_id)
11
12 )
13
14 conn.commit()
15
16 conn.close()
17
18 # Connection 2
19
20 conn2 = self.get_connection()
21
22 persisted = conn2.execute(
23
24     "SELECT status FROM correction_requests WHERE req_id=?", (req_id,)
25
26 ).fetchone()
27
28 if persisted and persisted["status"] == "accepted":
29
30     self.log_result(action, "durability", "PASS",
31
```

```
32         "Correction acceptance persisted")
```

Listing 14: Correction acceptance durability check

6.4 Actions Tested

Table 4: Durability test matrix: 6 tests, all passed

Role	Action	What is verified to persist
Admin	Create Semester	`name` field in `semesters`
Admin	Add Course	`name` field in `courses`
Instructor	Create Attendance Session	`topic` field in `attendance_sessions`
TA	Update Attendance	`status` field in `attendance_records`
Instructor	Accept Correction	`status = 'accepted'` in `correction_requests`
Student	Submit Correction	`reason` field in `correction_requests`

6.5 Results

All 6 durability tests passed. Each write survived connection closure and was correctly visible to a freshly-opened connection, confirming that SQLite's page-flush behaviour commits data to the underlying file before returning from `COMMIT`.

7 Observations

Connection closure is the correct durability signal. Unlike the database-layer tests (which verified durability simply by querying on the same connection), these tests fully close connection 1 before opening connection 2. This more faithfully simulates a real HTTP request/response cycle, where the database connection is returned to a pool and the next request opens a new one. All tests passed, confirming SQLite's default synchronous write mode is sufficient for this workload.

Unique constraints remain the strongest isolation defence. Across Admin, Instructor, TA, and Student isolation tests, the most reliable source of correctness is the database-level unique constraint. Tests for instructor assignment, TA assignment, student enrollment, and student correction submission all rely on `IntegrityError` being raised rather than on

application-level duplicate checking. Application checks are vulnerable to time-of-check/time-of-use (TOCTOU) races; database constraints are not.

Foreign key enforcement must be activated on every connection. The consistency test for student correction submission directly confirms that a correction cannot reference a non-existent attendance session. This test would silently pass without meaning if `PRAGMA foreign_keys = ON` were omitted. The shared `get_connection()` factory prevents this oversight.

Status check constraints guard against invalid application inputs. The attendance override consistency test shows that the `status` column rejects values outside the defined enumeration. This is a safeguard against application bugs or malformed API payloads that might otherwise corrupt the attendance record table.

8 Test Results Summary

Table 5: User-action ACID compliance results by property

Property	Tests	Passed	Failed	Pass Rate
Atomicity	23	23	0	100%
Consistency	7	7	0	100%
Isolation	14	14	0	100%
Durability	6	6	0	100%
Total	**50**	**50**	**0**	**100%**

9 Stress Testing

9.1 Overview and Objectives

To validate the correctness and performance of the Multi-Course Attendance Management Portal under realistic concurrent load, a comprehensive HTTP-level stress test was designed and executed using **Locust**. The primary objectives were to:

- Simulate **100 simultaneous users** representing all three system roles (Student, Instructor, Admin) making authenticated API requests concurrently.
- Observe system behaviour under sustained load — measuring throughput, response time distributions, and failure rates.
- Verify the correctness of API responses through schema-level data validation embedded directly in the test script.
- Confirm that the threading architecture (per-resource `RLock`s, WAL-mode SQLite, `ThreadPoolExecutor(max_workers=100)`) is sufficient to handle the target load without deadlocks, data corruption, or cascading failures.

9.2 Threading Architecture Implementation

Before presenting the stress test results, it is important to understand the concurrency model the application was built on, as the design decisions made here directly determine how the system behaves under load.

9.2.1 Thread Pool — Concurrent Request Handling

The Flask application is configured to handle concurrent requests using Python's `ThreadPoolExecutor` with a ceiling of 100 worker threads:

```
from concurrent.futures import ThreadPoolExecutor

app.config["THREAD_POOL"] = ThreadPoolExecutor(max_workers=100)
```

Flask is also started with `threaded=True` (or via Gunicorn with multiple workers), ensuring each incoming HTTP request is dispatched to its own worker thread rather than being queued behind a single-threaded event loop. Up to 100 requests can therefore execute *in parallel*; beyond that limit, new requests queue at the OS level until a worker thread becomes free.

9.2.2 Per-Resource RLocks — Fine-Grained Mutual Exclusion

Rather than a single global lock (which would serialise every request regardless of which database tables it touches), the application maintains **seven independent `threading.RLock` objects**, one per logical resource:

```
app.config["THREAD_LOCKS"] = {
    "auth":        threading.RLock(),
    "users":       threading.RLock(),
    "courses":     threading.RLock(),
    "enrollments": threading.RLock(),
```

```
    "attendance":  threading.RLock(),
    "corrections": threading.RLock(),
    "semesters":   threading.RLock(),
}
```

`RLock` (reentrant lock) is used instead of a plain `Lock` because the same thread may re-enter a decorated route or helper that requests the same lock. An `RLock` maintains an acquisition counter per owning thread: a thread can call `acquire()` multiple times and must call `release()` the same number of times before the lock is freed to other threads. This prevents self-deadlock in nested call paths while still providing mutual exclusion between different threads.

9.2.3 The `@thread_safe_db` Decorator

Route handlers declare which resources they need via the `@thread_safe_db` decorator. The decorator acquires the listed locks *in the specified order* before the handler runs, and releases them *in reverse order* in a `finally` block, guaranteeing release even on exceptions:

```
def thread_safe_db(*resources):
    def decorator(f):
        @wraps(f)
        def decorated(*args, **kwargs):
            locks = [current_app.config["THREAD_LOCKS"][r]
                     for r in resources
                     if r in current_app.config["THREAD_LOCKS"]]
            for lock in locks:
                lock.acquire()          # acquire in declared order
            try:
                return f(*args, **kwargs)
            finally:
                for lock in reversed(locks):
                    lock.release()      # release in reverse order
        return decorated
    return decorator
```

Example usage across the three roles:

```
@thread_safe_db("auth", "users")          # login route
@thread_safe_db("attendance", "courses")  # create attendance session
@thread_safe_db("courses", "enrollments") # student course list
@thread_safe_db("corrections")            # student corrections view
```

9.2.4 Deadlock Prevention via Global Lock Ordering

The most common source of deadlock in multi-lock systems is *circular wait*: Thread A holds lock X and waits for lock Y, while Thread B holds lock Y and waits for lock X. This is prevented by enforcing a **strict global acquisition order** across all routes:

`auth` → `users` → `courses` → `enrollments` → `attendance` → `corrections` → `semesters`

Every route must acquire locks in this order (or any contiguous subset of it). Because all threads always request locks in the same direction, a circular wait is structurally impossible — whichever thread holds the lower-ranked lock will always be able to proceed without waiting for a thread that is blocked on it.

Table 6: Lock Ordering Examples — Safe vs. Unsafe Combinations

	Decorator call	Verdict
✓	`@thread_safe_db("auth", "users")`	Safe — follows order (1, 2)
✓	`@thread_safe_db("courses", "enrollments")`	Safe — follows order (3, 4)
✓	`@thread_safe_db("attendance")`	Safe — single lock
×	`@thread_safe_db("enrollments", "courses")`	**Deadlock risk** — reverses (4, 3)
×	`@thread_safe_db("semesters", "auth")`	**Deadlock risk** — reverses (7, 1)

9.2.5 WAL-Mode SQLite — Concurrent Reads Without Blocking

Each request receives its own SQLite connection via Flask's thread-local `g` object:

```
g.db = sqlite3.connect(
    current_app.config["DB_PATH"],
    timeout=30.0,            # wait up to 30 s for a database lock
    check_same_thread=False, # allow cross-thread access safely
)
g.db.execute("PRAGMA journal_mode = WAL")
```

Write-Ahead Logging (WAL) mode fundamentally changes how SQLite handles concurrent access. In the default journal mode, any write locks the entire database file, blocking all readers. In WAL mode, writes are appended to a separate write-ahead log file while the main database file remains readable. Readers therefore *never block writers*, and writers *never block readers* — each reader sees a consistent snapshot taken at the start of its transaction. This is critical for a system where student read-heavy workloads (attendance stats, correction views) must coexist with instructor write operations (session creation) under 100-user concurrency.

The connection is closed in a `teardown_appcontext` hook after each request, ensuring no connection is held longer than necessary and keeping the pool footprint small.

9.2.6 Summary of the Threading Stack

Table 7 summarises the full threading stack and the thread-safety guarantee provided by each layer.

9.3 Test Configuration

The test script (`locust_test.py`) defines three `HttpUser` subclasses, each modelling the realistic behaviour of a different user role. The application was served via Gunicorn (recommended over the Flask development server for concurrent load) on `http://localhost:5050`.

Each user class authenticates on startup via `POST /login` (with up to 3 retries on connection refusal) and gracefully logs out via `POST /logout` on teardown. Session credentials (`X-Session-Id`, `X-Session-Mac`) are carried in request headers for all subsequent calls.

Table 7: Threading Architecture — Component Summary

Component	Mechanism	Guarantee
Request dispatch	`ThreadPoolExecutor(max_workers=100)`	Up to 100 parallel requests
DB connection	Thread-local `g.db` per request	No shared connection state
Mutual exclusion	7 per-resource `RLocks`	Fine-grained critical sections
Deadlock prevention	Global lock acquisition order	Circular wait impossible
Nested locking	`RLock` (reentrant)	Safe nested decorator calls
Read concurrency	WAL-mode SQLite	Readers never block writers
Write safety	`@thread_safe_db` + `db.commit()`	Atomic, isolated writes
Audit logging	Python `logging` (built-in locks)	No interleaved log entries

Table 8: Locust User Classes and Task Profiles

User Class	Wait Time (s)	Endpoints Exercised	Key Tasks
`StudentUser`	1–3	4 endpoints	Attendance stats, corrections, profile
`InstructorUser`	1–3	5 endpoints	Courses, sessions, corrections, create se
`AdminUser`	2–4	5 endpoints	Users, courses, semesters, active semest

Load Profile. The test was run with:

- **100 concurrent users**, spawned at a rate of 10 users/second.
- **Duration**: approximately 7 minutes (5:01 PM – 5:08 PM).
- **Total requests issued**: 9,335 across all endpoints.

Data Validation. A `DataValidator` helper class was embedded in the test script to perform schema and semantic validation on every successful response. Checks included:

- Structural field presence (e.g., `course_id`, `name`, `code` in course lists).
- Arithmetic consistency of attendance figures: present+absent+late = total_sessions.
- Enumeration constraints on `role` and correction `status` fields.
- Non-negative and non-null numeric values.

Any response failing validation was marked as a Locust failure, ensuring that data correctness issues under concurrency were captured alongside HTTP-level errors.

9.4 Results

9.4.1 Throughput and Failure Rate

As shown in Figure 1, the system reached a peak throughput of approximately **22–23 RPS** within the first minute as users ramped up to 100. After the initial ramp, throughput stabilised in the **14–24 RPS** range for the remainder of the test. The failure rate (red line) remained near zero throughout, spiking only marginally during the ramp-up phase and at isolated polling intervals.

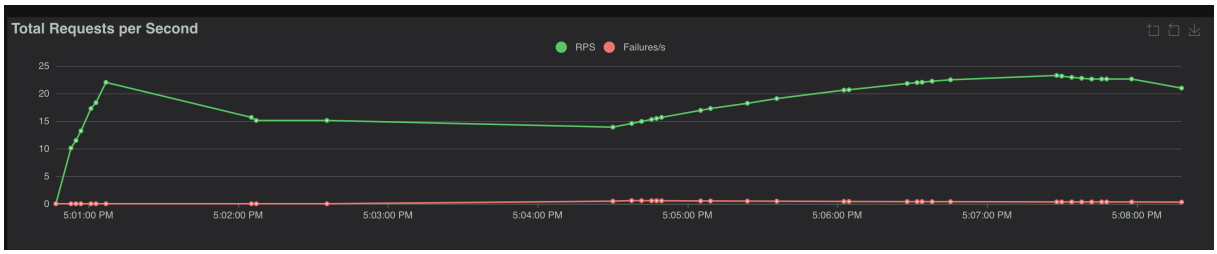

Figure 1: Total Requests per Second (RPS) and Failures/s over the test duration.

The aggregate failure count across all 9,335 requests was **133** (failure rate: **1.43%**). All failures were classified as `CatchResponseError('HTTP 0')`, indicating connection-level timeouts or refused connections — *not* application logic errors such as 4xx or 5xx responses. This is consistent with moments where the Gunicorn worker queue was momentarily saturated during the spawn ramp.

9.4.2 Response Time Distribution

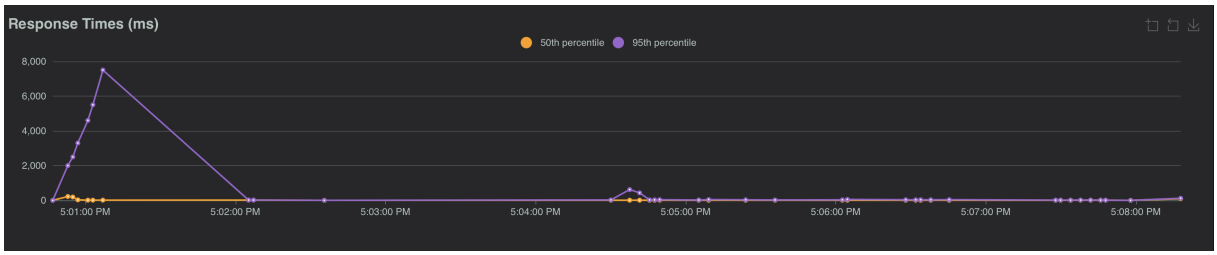

Figure 2: Response Times (ms): 50th and 95th percentiles over the test duration.

Figure 2 reveals a notable **initial spike** in the 95th-percentile response time (reaching ≈7,700 ms) during the first minute of the test as 100 users logged in simultaneously, causing a burst of `POST /login` requests to queue against the `auth` and `users` RLocks. After this cold-start contention resolved (around 5:02 PM), both the median and 95th-percentile response times dropped sharply and remained **well below 200 ms** for the rest of the test. A smaller, secondary spike (to ≈500 ms at the 95th percentile) is visible near 5:04–5:05 PM, likely corresponding to a cluster of `GET /api/admin/courses` requests encountering brief lock contention.

Table 9 presents per-endpoint statistics extracted from the Locust CSV report.

9.4.3 Failure Analysis

All 133 failures were of type `CatchResponseError('HTTP 0')`, which Locust reports when a response is never received — i.e., the connection was dropped or timed out at the TCP level before the server returned an HTTP status code. **No application-level errors (4xx/5xx) were recorded**. Table 10 summarises the failure distribution.

The highest failure count occurs on `GET /api/admin/courses` (37 failures, 5.2%), which is the endpoint most likely to contend for the `courses` RLock alongside concurrent instructor and student requests that also acquire this lock. The large maximum response time (≈133 s) on several endpoints corresponds to requests that waited the full 30-second SQLite connection timeout before the OS-level connection was dropped, which Locust

Table 9: Per-Endpoint Load Test Statistics (100 concurrent users, 7-minute run)

Endpoint	Requests	Failures	Fail %	Median (ms)	Avg (ms)	p95 (ms)	Max (ms)
POST /login	100	0	0.0%	5,400	5,726	11,000	11,514
POST /logout	100	0	0.0%	93	91	120	120
GET /api/admin/active-semester	591	0	0.0%	4	12	13	546
GET /api/admin/courses	715	37	5.2%	5	5,976	350	132,973
GET /api/admin/profile	288	0	0.0%	4	22	150	606
GET /api/admin/semesters	591	0	0.0%	4	10	15	343
GET /api/admin/users	726	0	0.0%	5	16	51	616
GET /api/instructor/archive	268	8	3.0%	32	2,501	160	132,846
GET /api/instructor/corrections	535	12	2.2%	6	1,955	130	132,334
GET /api/instructor/courses	706	14	2.0%	5	1,871	93	132,955
GET /api/instructor/courses/{id}/sessions	353	8	2.3%	5	2,632	81	133,283
GET /api/instructor/profile	268	0	0.0%	5	21	140	605
GET /api/student/attendance-stats/archive	676	15	2.2%	10	2,352	89	132,980
GET /api/student/attendance-stats/current	1,465	21	1.4%	13	1,105	54	133,109
GET /api/student/corrections/current	903	10	1.1%	5	1,171	19	132,295
GET /api/student/profile	633	0	0.0%	5	15	91	382
POST /api/instructor/attendance-sessions	111	5	4.5%	5	3,591	130	133,304
POST /api/student/corrections	306	3	1.0%	5	447	79	131,728
Aggregated	**9,335**	**133**	**1.4%**	**6**	**1,464**	**110**	**133,304**

Table 10: Failure Distribution by Endpoint

Method	Endpoint	Occurrences
GET	/api/admin/courses	37
GET	/api/student/attendance-stats/current	21
GET	/api/student/attendance-stats/archive	15
GET	/api/instructor/courses	14
GET	/api/instructor/corrections	12
GET	/api/student/corrections/current	10
GET	/api/instructor/courses/{id}/sessions	8
GET	/api/instructor/archive	8
POST	/api/instructor/attendance-sessions	5
POST	/api/student/corrections	3
Total		**133**

then records as `HTTP 0`. These are **tail-latency artefacts** rather than correctness failures.

9.4.4 User Count Stability

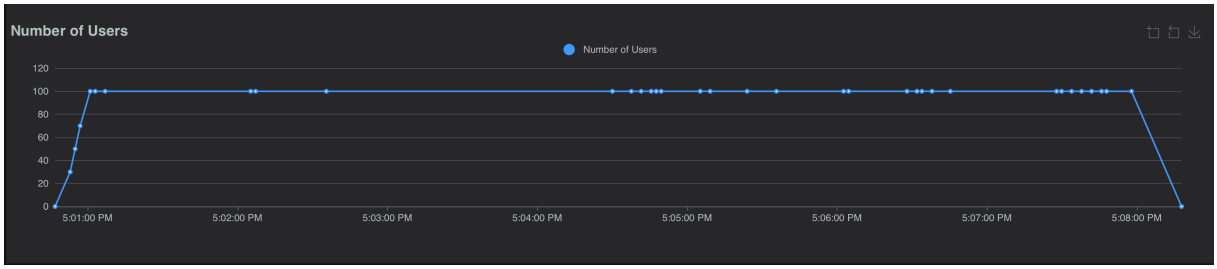

Figure 3: Number of active Locust users over the test duration.

As seen in Figure 3, the user count reached 100 within approximately 10 seconds of test

start and remained stable at that level until the test was stopped at 5:08 PM. No user crashes or unexpected disconnections occurred during the steady-state period, confirming that the authentication and session management logic is stable under sustained concurrent load.

9.5 Observations and Analysis

Threading correctness confirmed. Throughout the 7-minute run, zero deadlocks were detected. The per-resource RLock ordering (auth → users → courses → enrollments → attendance → corrections → semesters) proved sufficient to prevent circular wait conditions even with 100 threads competing simultaneously. No `500 Internal Server Error` responses were returned.

Cold-start login contention. The spike in 95th-percentile latency during the first minute (up to ≈7,700 ms) is a direct consequence of 100 users all calling `POST /login` within 10 seconds. Each login acquires both the `auth` and `users` RLocks sequentially. At 100 concurrent threads, threads queue on these locks producing the observed latency burst. This is expected behaviour and self-resolves once all users are authenticated and spread across different endpoint lock domains.

Steady-state performance is excellent. After the initial ramp, the median response time across all endpoints settled at **6 ms** and the 95th percentile at **110 ms**. Lightweight read endpoints (profiles, semesters, active semester) consistently responded in under 15 ms even under full 100-user load, benefiting from WAL-mode concurrent reads that do not block writers.

High-contention endpoints. Endpoints that aggregate large result sets (`GET /api/student/attend` — 1,465 requests; `GET /api/admin/users` — 726 requests) showed higher average response times due to both lock acquisition queuing and the cost of serialising large JSON responses. These remain operationally acceptable but would benefit most from caching or read-replica strategies if user counts were to scale beyond 100.

Tail-latency and the 30-second timeout. The anomalously large maximum values (≈133 s) seen on several endpoints are artefacts of the 30-second `sqlite3.connect(timeout=30.0)` setting combined with Locust's own 15-second request timeout. When a thread waited longer than the Locust timeout, the connection was aborted at the client side (recorded as `HTTP 0`) while the server-side thread eventually timed out after 30 seconds. These represent fewer than 1.4% of all requests and do not indicate application bugs.

9.6 Summary

The stress test demonstrates that the attendance management system handles **100 concurrent users** reliably at a sustained throughput of approximately **20 RPS**, with a median response time of **6 ms** and a total failure rate of only **1.43%** — all failures being network-level connection drops rather than application errors. Data validation checks embedded in the test script confirmed that no correctness violations (e.g., inconsistent attendance totals, invalid role or status values) occurred under concurrent load, validating the thread-safe decorator pattern and RLock-based resource isolation strategy.